# 後端開發工程師課程 — 專題實作作業說明書

Oct 24, 2026 · by Gary Chang

## 一、專題目標與定位

這份專題要驗收的是「能不能獨立交付一個雲原生的 AI 後端系統」，而不是個別技術的熟練度。

課程內容涵蓋：Phase 1（42hr）建立 Maven、測試、Spring Boot 三層架構、JPA、REST API、Spring Security 的開發基礎；Phase 2（66hr）加上容器化、微服務模式、OAuth2/OIDC、Kubernetes、CI/CD、Spring AI 與向量資料庫。專題要求把這兩階段的能力在同一個系統裡串起來。

具體要驗收三件事：

1. **架構判斷力**：服務邊界怎麼切、哪裡該用異步、哪裡該讀寫分離，並能說出取捨理由。會用不等於用得對。
2. **工程完整度**：從資料驗證、例外處理、測試、API 文件到一鍵啟動的容器化環境，缺一項就不是可交付的系統。
3. **AI 整合能力**：Spring AI 不是加一個聊天視窗，而是讓 AI 真正參與業務流程（結構化擷取、工具呼叫、基於企業知識的問答），並處理 AI 失敗時的降級路徑。

完成後的成果可直接作為求職作品集，README 與架構決策文件是面試時最常被追問的部分，請認真寫。

## 二、選題

本專題不分組，每人獨立完成自己的系統。

**選題**：從下列五題選一。同一題目可有多人選擇，但各自設計與實作，不得共用程式碼。若自訂題目，須先提一頁簡要說明（涵蓋那些技術重點)。

| 題目 | 一句話情境 | 技術重心 | 難度 |
| --- | --- | --- | --- |
| 1. 線上課程學習平台 | 講師開課、學員報名與記錄進度，附教材 AI 助教 | 課程排行榜的讀寫分離、教材 RAG 的 Metadata 權限過濾 | 低 |
| 2. 智慧 IT 服務工單系統 | 員工開單、工程師處理、主管看 SLA，AI 從知識庫建議解法 | 狀態機驗證、工單自動分類的結構化輸出、對話式工具呼叫 | 中高 |
| 3. 診所預約與看診管理系統 | 病人線上掛號、醫師排班、櫃台報到 | 同時搶掛的併發控制、水平越權防護、衛教問答的 System Prompt 邊界 | 中 |

| 題目 | 一句話情境 | 技術重心 | 難度 |
| --- | --- | --- | --- |
| 4. 餐飲外送平台 | 餐廳上架菜單、顧客下單、外送員接單 | 訂單狀態事件流、簡化版 Saga 補償、多函式協作點餐 | 高 |
| 5. 企業差旅報銷與簽核系統 | 員工上傳收據、主管多級簽核、財務撥款 | 收據內容結構化擷取與降級、差旅政策 RAG 的條文引用 | 中高 |

各題的完整情境說明、服務拆分建議與實作重點，見 `專題題目說明` 分頁。

**重要**：請務必避開「線上書店」與「線上購物商城」——這兩個已在課堂練習與範例中使用過。

## 三、共同基本要求（必做）

不論選哪一題，下列七個面向都要做到。沒有指定特定元件的部分，可自行選技術，但須在架構決策文件說明理由。

| 面向 | 必須涵蓋 |
| --- | --- |
| 架構 | 至少 3 個業務微服務，並**實際運用微服務的重要模式**（如 CQRS 讀寫分離、RabbitMQ 異步流程、API Gateway、服務發現、斷路器等），須在文件中說明為何選用這些模式 |
| 資料 | JPA 關聯映射（含一對多 / 多對多）、分頁排序、`@Transactional`、Auditing |
| 安全 | Keycloak 作 Authorization Server，API 為 Resource Server；服務間用 Client Credentials；至少一個第三方登入（GitHub / Google） |
| 品質 | DTO + Bean Validation + 全域例外處理 + SpringDoc；單元測試（Mockito）＋整合測試（MockMvc / `@DataJpaTest`），覆蓋率目標 60% |
| 部署 A：Docker Compose（二擇一，10 分） | Multi-stage Dockerfile、`docker compose up` 一鍵啟動完整環境（含 DB / RabbitMQ / Chroma / Keycloak）、健康檢查與 Volume 資料持久化 |

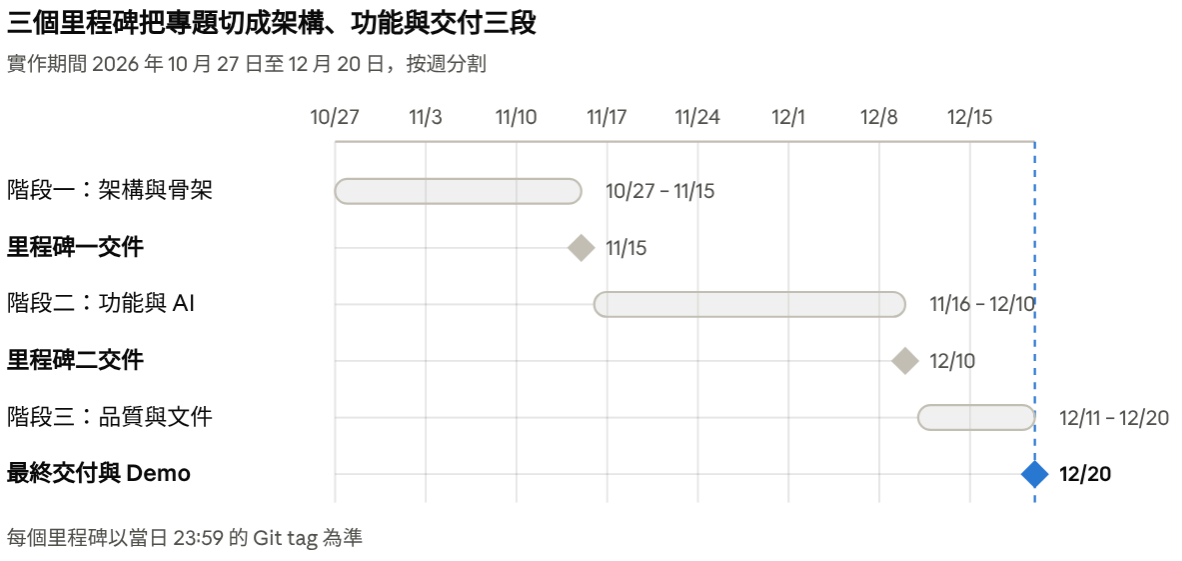
| 面向 | 必須涵蓋 |
| --- | --- |
| 部署 B：Kubernetes 基礎部署（二擇一，15 分） | 撰寫 manifests（Deployment / Service / Ingress / ConfigMap / Secret / PVC / 健康探針），部署至本地叢集（Docker Desktop、minikube 或 kind）並跑通主流程 |
| AI | Spring AI：Chat + PromptTemplate + Structured Output + Function Calling + RAG（Chroma 持久化向量庫） |

兩點說明：

- 「架構」欄不指定元件，是因為 K8s Service DNS 可以取代 Eureka、Ingress 可以取代 Spring Cloud Gateway，這本身就是值得思辨的架構題。選什麼不是重點，說得出理由才是。
- AI 功能必須與業務流程相關。只是外接一個通用聊天機器人，AI 面向最多給一半分數。
- 部署 A 與部署 B 二擇一即可，但配分不同：做到 Docker Compose 一鍵啟動為 10 分，改以 Kubernetes 部署至本地叢集為 15 分。選 B 者仍須提供 Multi-stage Dockerfile 與可用的映像檔建置方式，但不必再維護一套 Compose 環境。

## 四、三階段里程碑

三個里程碑依序為「架構立得起來」、「功能與 AI 跑得通」、「整個系統交得出去」。每個里程碑都要在指定日期 23:59 前 push 至 GitHub 並打上 tag（`m1` / `m2` / `m3`），逃過一個里程碑即使最後做完也扣 5 分。




三階段里程碑 · 11/15、12/10、12/20

第二階段服務四週，是工作量最重的一段；最後十天只留給補文件與修bug，不要把功能拖到這裡做。

## 里程碑一：2026-11-15 — 架構與骨架

**目標**：服務邊界定案，完整環境能一鍵啟動，資料層跟得上。此階段不要追功能數量，要能「跑得起來的骨架」。

交付內容：

1. 架構決策文件 v1：系統情境說明、服務拆分與理由、ER 與 API 草案、打算用哪些微服務模式
2. Monorepo 或多 repo 結構建立，至少 2 個服務能啟動並回應健康檢查端點
3. `docker-compose.yml` 能一鍵起服務與 DB，資料以 Volume 持久化
4. 核心 Entity 與 Repository 完成，含至少一組關聯映射，有可用的測試資料

**檢核方式**：助教 clone 後執行 `docker compose up`，檢查服務是否啟動、健康端點是否回 200。課堂上 10 分鐘架構簡報，教師當場回饋服務邊界切得合不合理。

## 里程碑二：2026-12-10 — 功能與 AI 整合

**目標**：主流程從頭到尾跑得通，安全、異步與 AI 功能都已實際上線。這是工作量最重的階段。

交付內容：

1. 主要業務流程完整可操作（以工單系統為例：開單 → 指派 → 處理 → 結單）
2. 至少一條異步流程上線，含消費失敗的處理策略
3. 至少一個微服務模式落地並能 Demo（如 CQRS 查詢端、Gateway 統一入口）
4. Keycloak 整合完成：登入發 Token、API 驗證、至少兩種角色權限區隔
5. Spring AI 四項都有雛形：Chat、PromptTemplate、Structured Output、Function Calling
6. RAG 流程跑通：文件入庫與檢索問答，向量資料已持久化至 Chroma
7. Swagger UI 可瀏覽主要端點，測試覆蓋率至少 40%

**檢核方式**：課堂上 10 分鐘主流程 Demo，必須包含一次失敗情境（驗證失敗或權限不足），讓教師看錯誤回應格式。AI 功能要 Demo 至少一個 Structured Output 與一個 Function Calling。

## 里程碑三：2026-12-20 — 最終交付

**目標**：補文件與修bug，交出一個別人能照著 README 跑起來的系統。

交付內容：

1. 全部必做項目完成，測試覆蓋率達 60%
2. 架構決策文件 v2：涵蓋實際做下來的心得與踩到的坑

3. RAG 效果比較報告：至少 5 組問答的有 / 無 RAG 對照
4. README 完成：架構圖、啟動步驟、環境變數說明、API 清單
5. 部署成果（部署 A 或 B）並附驗證紀錄
6. 15 分鐘 Demo 與口頭答辯

**檢核方式**：助教依「七、驗收檢查清單」逐項打勾，Demo 後教師按評分表計分。

## 五、最終交付物清單

截止 2026-12-20 23:59，以 GitHub repo 的 `m3` tag 為評分依據。

1. **GitHub repo**（公開或邀請教師與助教），含完整 commit 歷程——最後一次大 commit 把所有程式丟上去者視為協作紀錄不足
2. **README.md**：架構圖、啟動步驟（含所需環境變數與 API Key 設定方式）、服務與端口表、主要 API 清單
3. **架構決策文件**（`docs/architecture.md`）：服務邊界理由、採用哪些微服務模式與為何、同步 / 異步的取捨、資料一致性策略、做得不好的地方與改進方向
4. **`docker-compose.yml`** 可一鍵啟動全部服務；若有做 K8s，附 `k8s/` manifests
5. **測試與文件證據**：Swagger UI 截圖、覆蓋率報告（JaCoCo）
6. **RAG 效果比較報告**（`docs/rag-eval.md`）：至少 5 組問答的有 / 無 RAG 對照，並說明 chunk 大小、Top-K 與相似度閾值怎麼調的
7. **Demo**：15 分鐘現場演示，無須投影片，直接跑系統

## 六、評分表

總分上限 100 分（部署選 A 者該面向上限 10 分，總分上限 95 分）。專題佔學期成績比重由教師在課程說明中公告。

| 面向 | 配分 | 評分細項 |
| --- | --- | --- |
| 架構正確性與取捨說明 | 25 | 服務邊界是否合理（不是硬切成三個）、微服務模式用得對不對（是否硬套）、能不能說清楚為何這樣選。口頭答辯的表現佔本面向一半 |
| 功能完整度 | 20 | 主流程是否從頭到尾可用、邊界情況有沒有處理、業務規則（狀態轉移、權限、併發）是否守得住 |
| AI 整合品質 | 20 | Spring AI 四項是否都落到業務上；RAG 是否引用來源、Metadata Filtering 是否正確；AI 失敗時有沒有降級路徑。純外接通用聊天機器人最多給一半 |

| 面向 | 配分 | 評分細項 |
| --- | --- | --- |
| 部署（Compose ＋ K8s） | 10 或 15 | `docker compose up` 是否真的一鍵跑得起來、Dockerfile 有沒有做 multi-stage、選部署 A 者以一鍵啟動驗收，上限 10 分；選部署 B 者另看 K8s manifests 能否部署至本地叢集並跑通主流程，上限 15 分 |
| 程式品質與測試 | 15 | 分層是否乾淨、DTO 與 Entity 是否分離、例外處理是否統一、測試是否測在有意義的地方（不是湊覆蓋率） |
| 文件與 Demo | 5 | README 能不能讓別人跑起來、架構圖是否符合實際程式、Demo 是否準時且流暢 |

**評等參考**（以架構面向為例，其餘面向同理類推）：

- **優（85～100%）**：邊界切法有業務依據，模式選用有明確動機，能主動指出自己設計的弱點與替代方案
- **良（70～84%）**：架構合理且跑得通，但取捨說明停在「課程教的就這樣做」
- **可（60～69%）**：模式有用到但有硬套痕跡（如無必要的 CQRS），亦無法說明理由
- **不及格（< 60%）**：服務拆分流於形式（如三個服務共用同一張表、互相直接讀對方資料庫）

## 七、驗收檢查清單

助教在最終驗收時逐項打勾；學生也請在交件前自檢一遍。

**環境與部署**

- [ ] `git clone` 後依 README 步驟，30 分鐘內能把系統跑起來
- [ ] `docker compose up` 一次成功，所有容器進入 healthy
- [ ] 容器重啟後資料不遺失（DB 與 Chroma 都有 Volume）
- [ ] Dockerfile 有 multi-stage，最終映像檔未包含建置工具
- [ ] 沒有任何 API Key、密碼或憑證被 commit 進 repo

**API 與資料層**

- [ ] Swagger UI 可開啟，主要端點都有說明與範例
- [ ] 驗證失敗時回應 400 並含欄位層級的錯誤訊息，格式全站一致
- [ ] 找不到資料回 404、權限不足回 403，不是一律 500
- [ ] 清單端點有分頁與排序，回應含總數與頁數
- [ ] 回應的是 DTO 而非 Entity（沒有意外外洩的欄位或循環引用）
- [ ] 關聯查詢沒有 N+1（能說明怎麼驗證的）

**微服務模式**

- [ ] 至少 3 個業務服務，各有自己的資料邊界（沒有跳過服務直接讀別人資料庫）
- [ ] 異步流程能現場 Demo，並能展示消費失敗時訊息去哪裡
- [ ] 宣稱用的模式在程式裡找得到，且文件說明了為何選它
- [ ] 能回答「這裡為何不用同步呼叫」與「資料不一致時會發生什麼」

### 安全

- [ ] 未帶 Token 存取受保護端點回 401
- [ ] 換成別人的 ID 無法讀到別人資料（水平越權已防護）
- [ ] 角色權限區隔有效，至少兩種角色行為不同
- [ ] 第三方登入可成功登入並自動建立本地帳號
- [ ] 服務間呼叫有帶 Token（不是內網就裸奉）

### AI 與 RAG

- [ ] Structured Output 回傳真正的 Java 物件，並有欄位驗證
- [ ] Function Calling 有可呼叫範圍限制與參數驗證
- [ ] RAG 回答有引用來源（文件名、條文或 ID）
- [ ] Metadata Filtering 正確，檢索不會跨越權限邊界
- [ ] AI 服務的超時、解析失敗有處理，不會把整個請求帶崩
- [ ] 向量資料已持久化，重啟後不須重建索引

### 品質與工程紀律

- [ ] 測試覆蓋率報告達 60%，且測試集中在 Service 層業務邏輯
- [ ] commit 歷程看得出漸進開發，不是最後一次倒入
- [ ] 架構圖與實際程式一致（沒有圖上有但程式裡沒的服務）

### K8s 本地部署

- [ ] K8s manifests 可部署至本地叢集並跑通主流程
- [ ] 健康探針、ConfigMap / Secret、PVC 都有實際生效

## 八、注意事項與常見扣分項

### 常見扣分項

- **AI 功能與業務脈絡脫鉤**：在系統角落接一個通用聊天室，跟業務資料沒關係。AI 面向最多給一半
- **模式硬套**：為了湊分數加 CQRS，卻只有一個查詢端且跟寫入端共用同一張表。寧可少做一個模式，把一個做正確
- **假微服務**：切成三個服務但共用同一個 schema，或服務 A 直接連服務 B 的資料庫
- **錯誤處理空洞**：正常流程很漂亮，但輸入一筆錯資料就回 500 與一大串 stack trace

- **AI 回應未驗證**：直接把 LLM 回傳的內容寫進資料庫，沒做欄位驗證，也沒有解析失敗的降級路徑
- **測試湊數字**：大量 getter/setter 測試拿覆蓋率，Service 層業務邏輯卻沒測
- **README 跑不起來**：助教照著做卻啟不動，沒寫所需環境變數或漏了初始資料
- **Demo 超時**：15 分鐘內要做完，請先演練。
- **憑證進版控**：API Key、資料庫密碼或 Keycloak client secret 被 commit 進 repo。一律用環境變數或 GitHub Secrets，.env 要進 .gitignore

### 對 AI 輔助開發的立場

歡迎用 Claude Code、Copilot 、Opencode + Ollama或其他工具幫忙，這就是實務。